

Autoencoders and Representation Learning

Courtesy

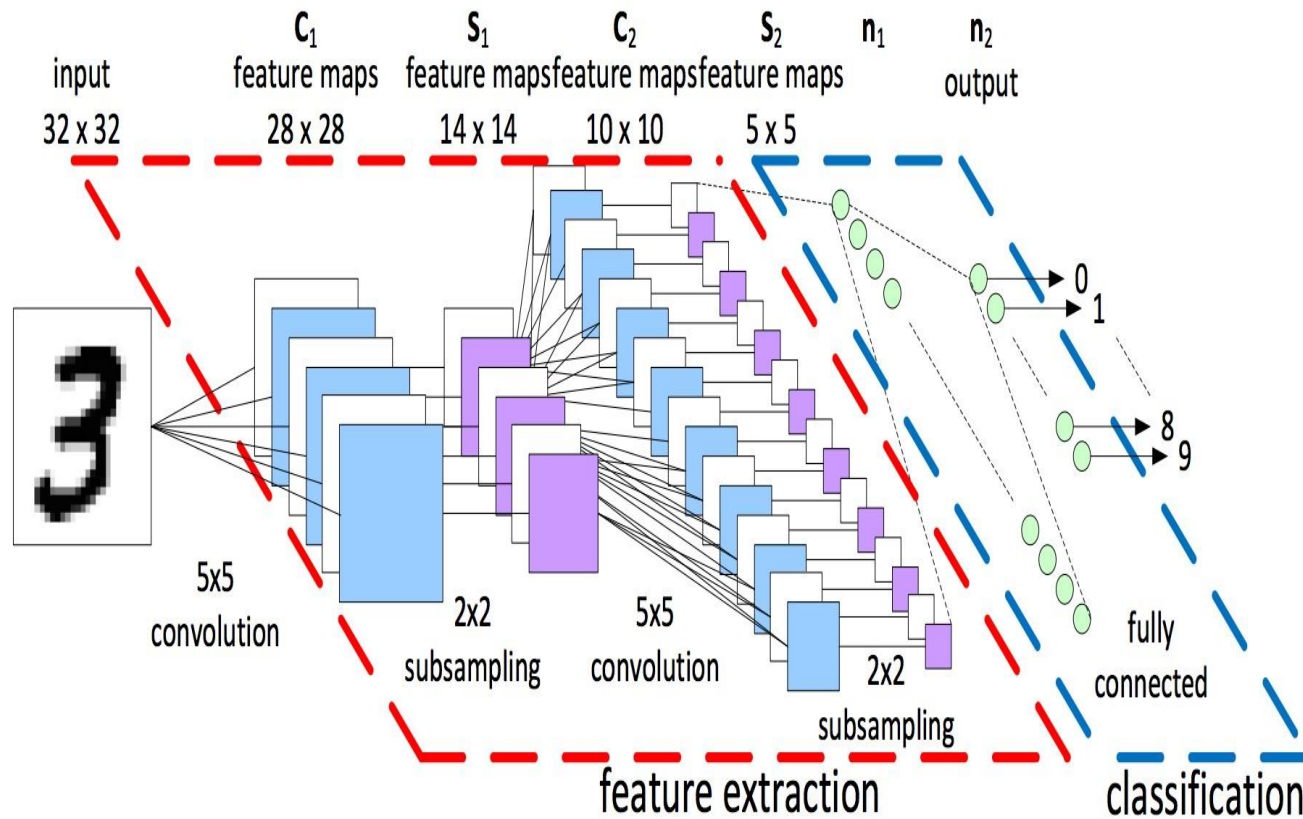
- Cloud & Lab
- Machine Learning at Berkeley

Review: Typical Neural Net Characteristics

So far, Deep Learning Models have things in common:

- Input Layer: (maybe vectorized), quantitative representation
- Hidden Layer(s): Apply transformations with nonlinearity
- Output Layer: Result for *classification, regression, segmentation, etc.*
- Models used for **supervised learning**

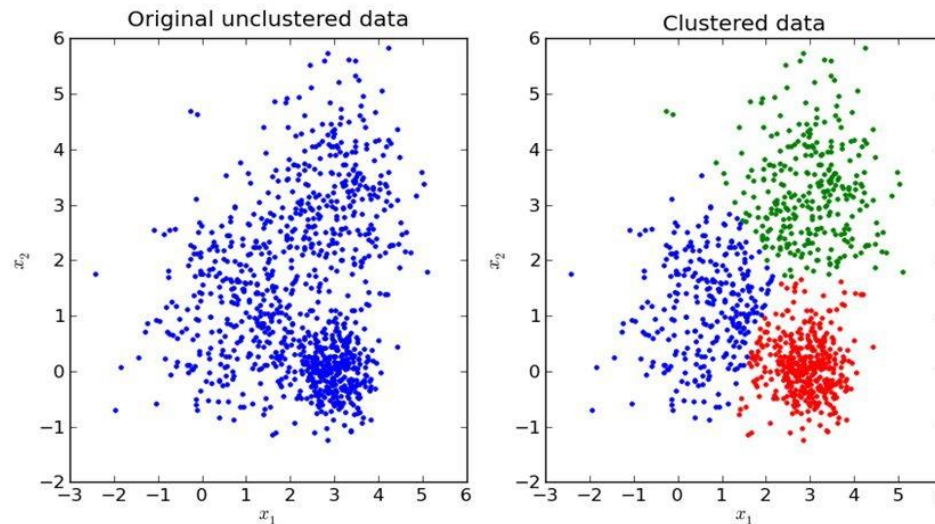
Example using Diagram



Changing the Objective

- Today's lecture: **unsupervised learning** with neural networks.

Unsupervised Learning



Autoencoders

- Autoencoders are
 - Artificial neural networks
 - Capable of **learning efficient representations** of the input data, called codings, without any supervision
 - The training set is unlabeled.
- These codings typically have a much **lower dimensionality** than the input data, making autoencoders useful for dimensionality reduction

Autoencoders

Why use Autoencoders?

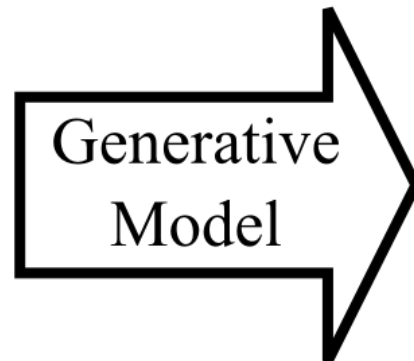
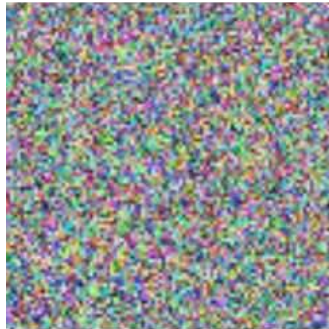
- Useful for dimensionality reduction
- Autoencoders act as powerful feature detectors,
- And they can be used for unsupervised pre-training of deep neural networks
- Lastly, they are capable of randomly generating new data that looks very similar to the training data; this is called a **generative model**.

Autoencoders

Why use Autoencoders?

For example, we could train an autoencoder on pictures of faces, and it would then be able to generate new faces

Noise $\sim N(0,1)$



Autoencoders

- Surprisingly, autoencoders work by simply learning to copy their inputs to their outputs
- This may sound like a trivial task, but we will see that constraining the network in various ways can make it rather difficult

Autoencoders

For example

- You can limit the size of the internal representation, or you can add noise to the inputs and train the network to recover the original inputs.
- These constraints prevent the autoencoder from trivially copying the inputs directly to the outputs, which forces it to learn efficient ways of representing the data
- In short, the codings are byproducts of the autoencoder's attempt to learn the identity function under some constraints

Autoencoders

What we'll learn?

- We will explain in more depth how autoencoders work
- What types of constraints can be imposed
- And how to implement them, whether it is for
 - Dimensionality reduction,
 - Feature extraction,
 - Unsupervised pretraining,
 - Or as generative models

Efficient Data Representations

Let's try to understand

**“why constraining an
autoencoder during training pushes it to discover
and exploit patterns in the data”**

With an example

Efficient Data Representations

**Which of the following number sequences do you find
the easiest to memorize?**

- 40, 27, 25, 36, 81, 57, 10, 73, 19, 68
- 50, 25, 76, 38, 19, 58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20

Efficient Data Representations

- At first glance, it would seem that the first sequence should be easier, since it is **much shorter**
- However, if you look carefully at the second sequence, you may notice that it follows two simple rules:
 - Even numbers are followed by their half,
 - And odd numbers are followed by their triple plus one

This is a famous sequence known as the **hailstone sequence**

Efficient Data Representations

- Once you notice this pattern, the second sequence becomes much easier to memorize than the first because you only need to memorize the two rules,
 - The first number,
 - And the length of the sequence

Efficient Data Representations

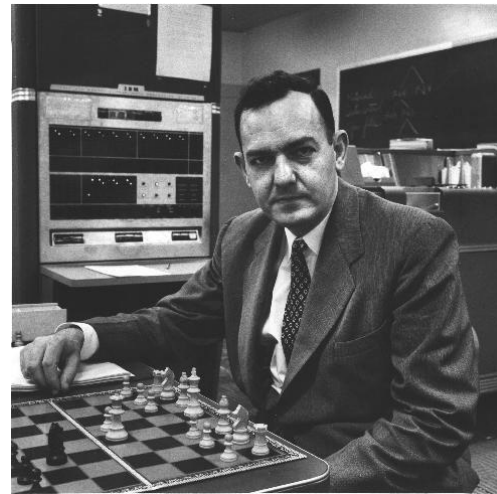
- Note that if we could quickly and easily memorize very long sequences, you would not care much about the **existence of a pattern** in the second sequence
- You would just learn every number by heart, and that would be that
- It is the fact that it is **hard to memorize long sequences** that makes it useful to **recognize patterns**,
- And hopefully this clarifies **why constraining an autoencoder during training pushes it to discover and exploit patterns in the data**

Efficient Data Representations

The relationship between memory, perception, and pattern matching was famously studied by **William Chase and Herbert Simon** in the early 1970s



William Chase



Herbert Simon

Efficient Data Representations

Let's see how their study of chess players is similar to an Autoencoder

- They observed that expert chess players were able to memorize the positions of all the pieces in a game by looking at the board for just **5 seconds**,
- A task that most people would find impossible
- However, this was only the case when the pieces were placed in realistic positions from actual games, not when the pieces were placed randomly.

Efficient Data Representations

Let's see how their study of chess players is similar to an Autoencoder

- Chess experts don't have a much better memory than you and I,
- They just see chess patterns more easily thanks to their experience with the game
- Noticing patterns helps them store information efficiently

Efficient Data Representations

Let's see how their study of chess players is similar to an Autoencoder

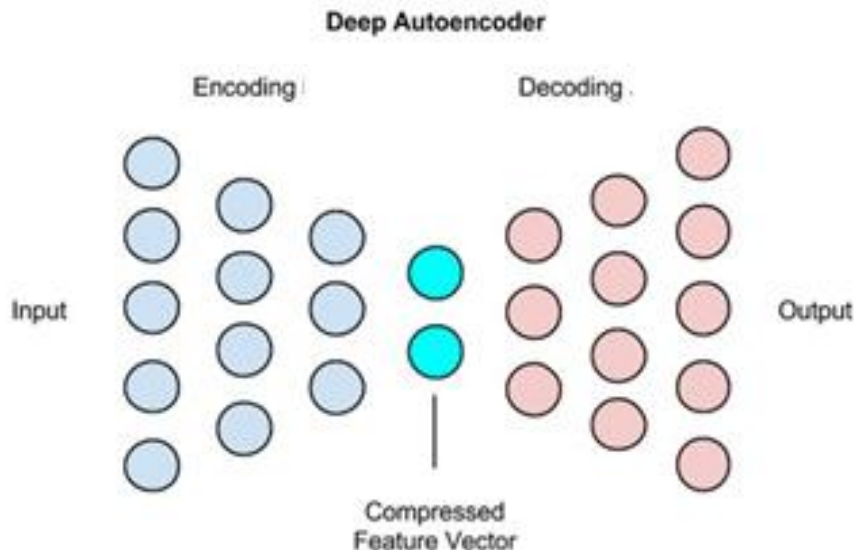
- Just like the chess players in this memory experiment, **an autoencoder**
 - looks at the inputs,
 - converts them to an efficient internal representation,
 - and then spits out something that (hopefully) looks very close to the inputs

Efficient Data Representations

Composition of Autoencoder

An autoencoder is always composed of **two parts**:

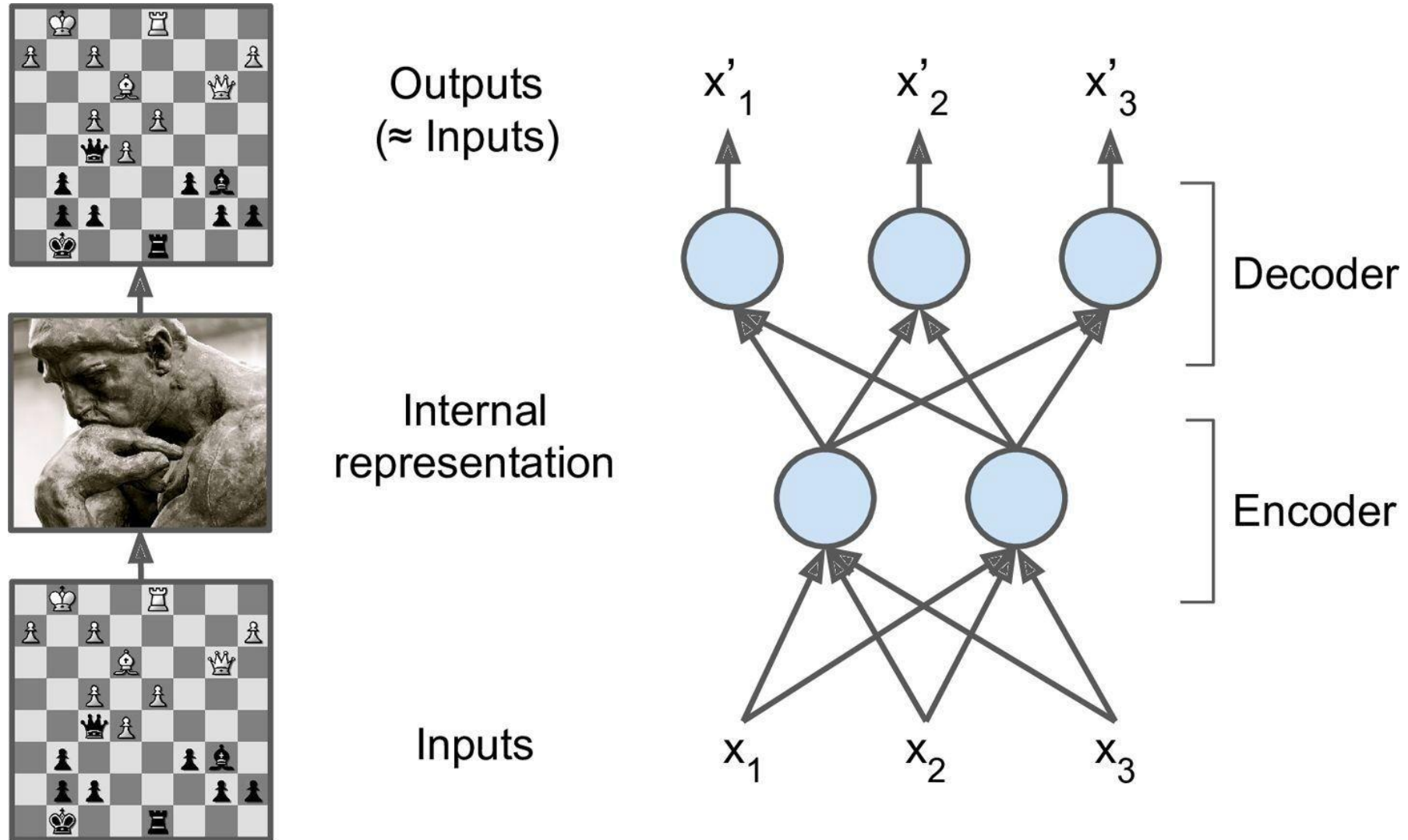
- **An encoder** or **recognition network** that converts the inputs to an internal representation,
- Followed by **a decoder** or **generative network** that converts the internal representation to the outputs



- Structure is almost always organized into **encoder** network, $\mathbf{f}$, and **decoder** network, $\mathbf{g}$: $model = \mathbf{g}(\mathbf{f}(\mathbf{x}))$
- Trained by gradient descent with **reconstruction loss**: measures differences between input and output e.g. MSE :
$$J(\theta) = |\mathbf{g}(\mathbf{f}(\mathbf{x})) - \mathbf{x}|^2$$

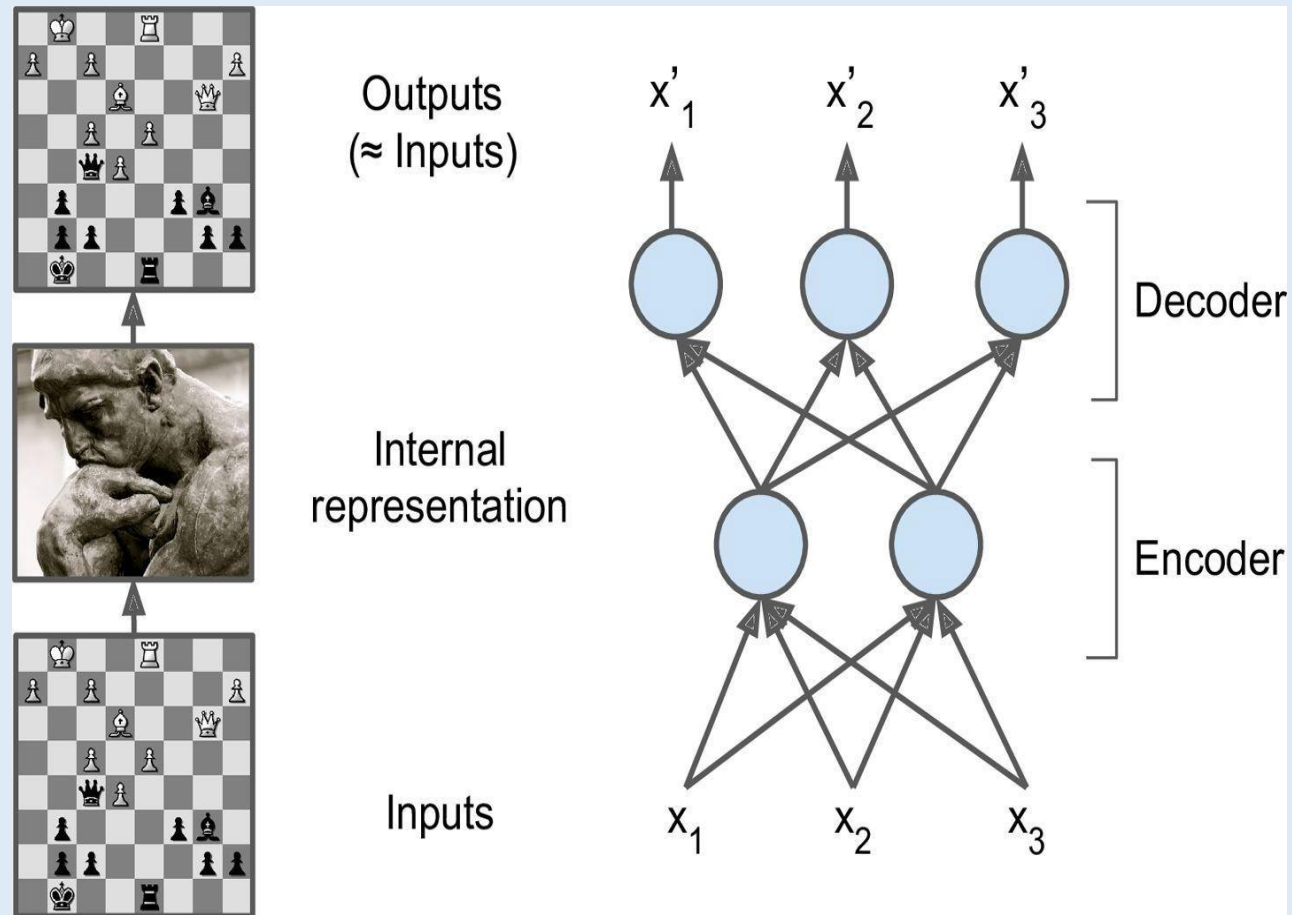
Efficient Data Representations

Composition of Autoencoder



Efficient Data Representations

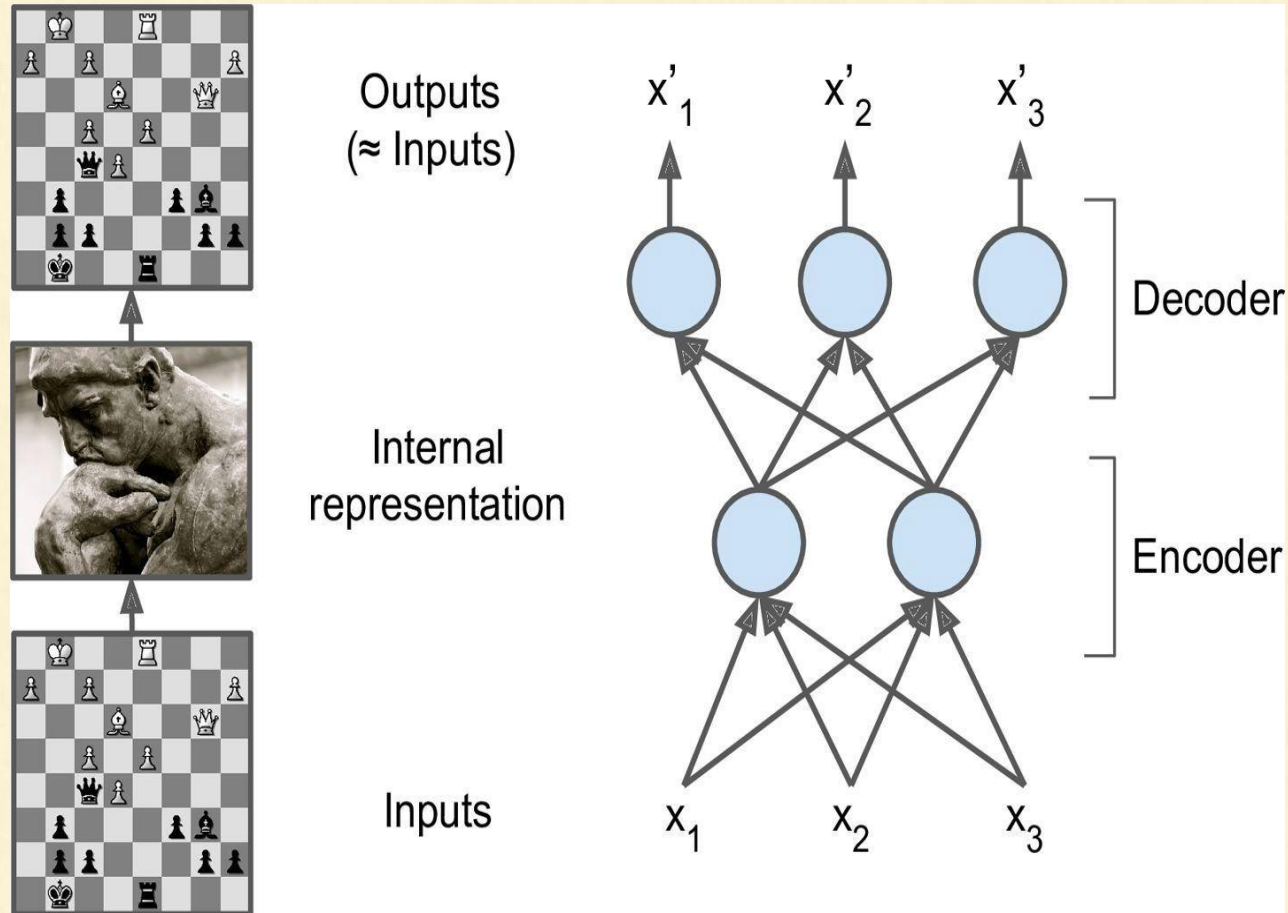
Composition of Autoencoder



An **autoencoder** typically has the same architecture as a **Multi-Layer Perceptron**, except that the **number of neurons** in the output layer must be **equal to the number of inputs**

Efficient Data Representations

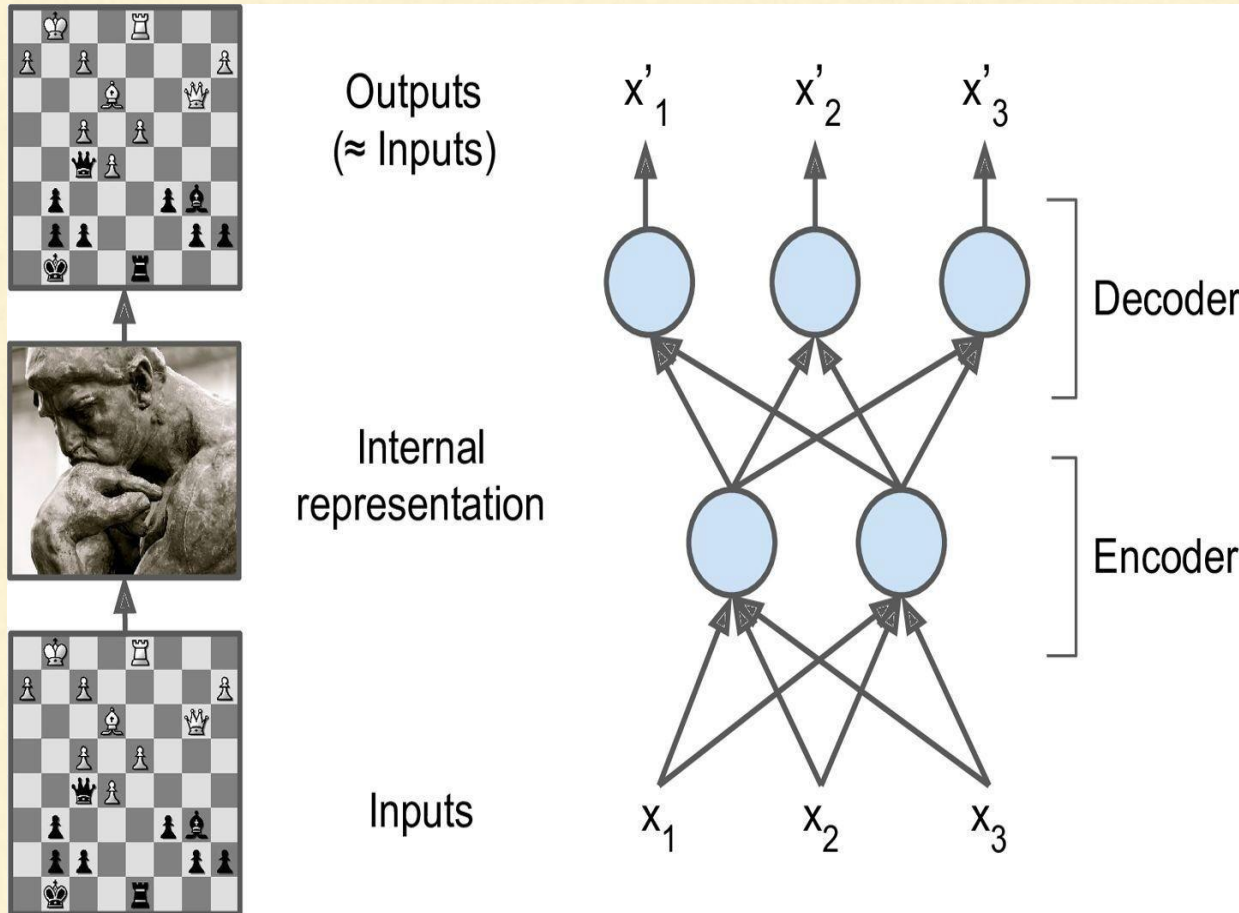
Composition of Autoencoder



There is just one hidden layer composed of two neurons **(the encoder)**, and **one output layer** composed of three neurons **(the decoder)**

Efficient Data Representations

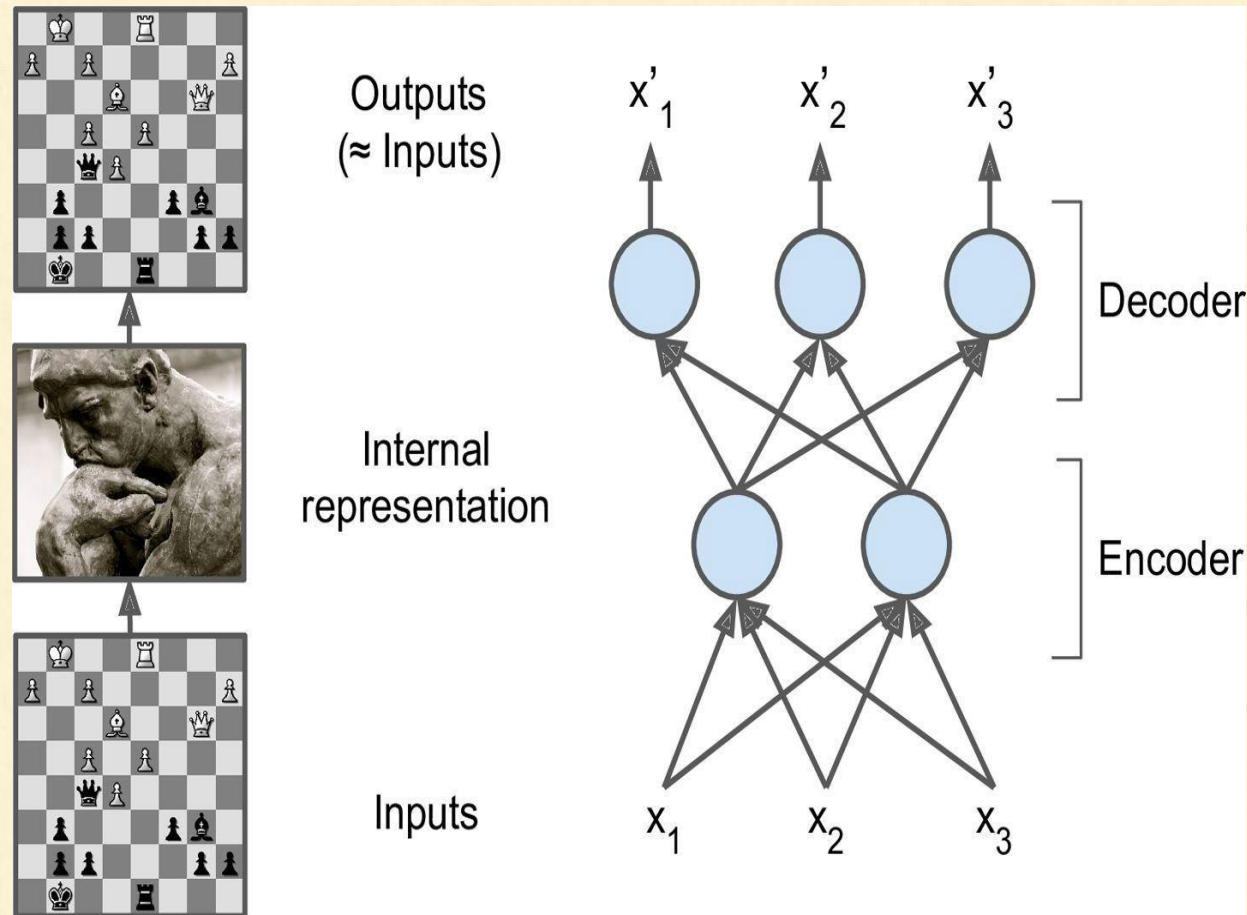
Composition of Autoencoder



The outputs are often called **the reconstructions** since the autoencoder tries to reconstruct the inputs, and the cost function contains a **reconstruction loss** that penalizes the model when the reconstructions are different from the inputs.

Efficient Data Representations

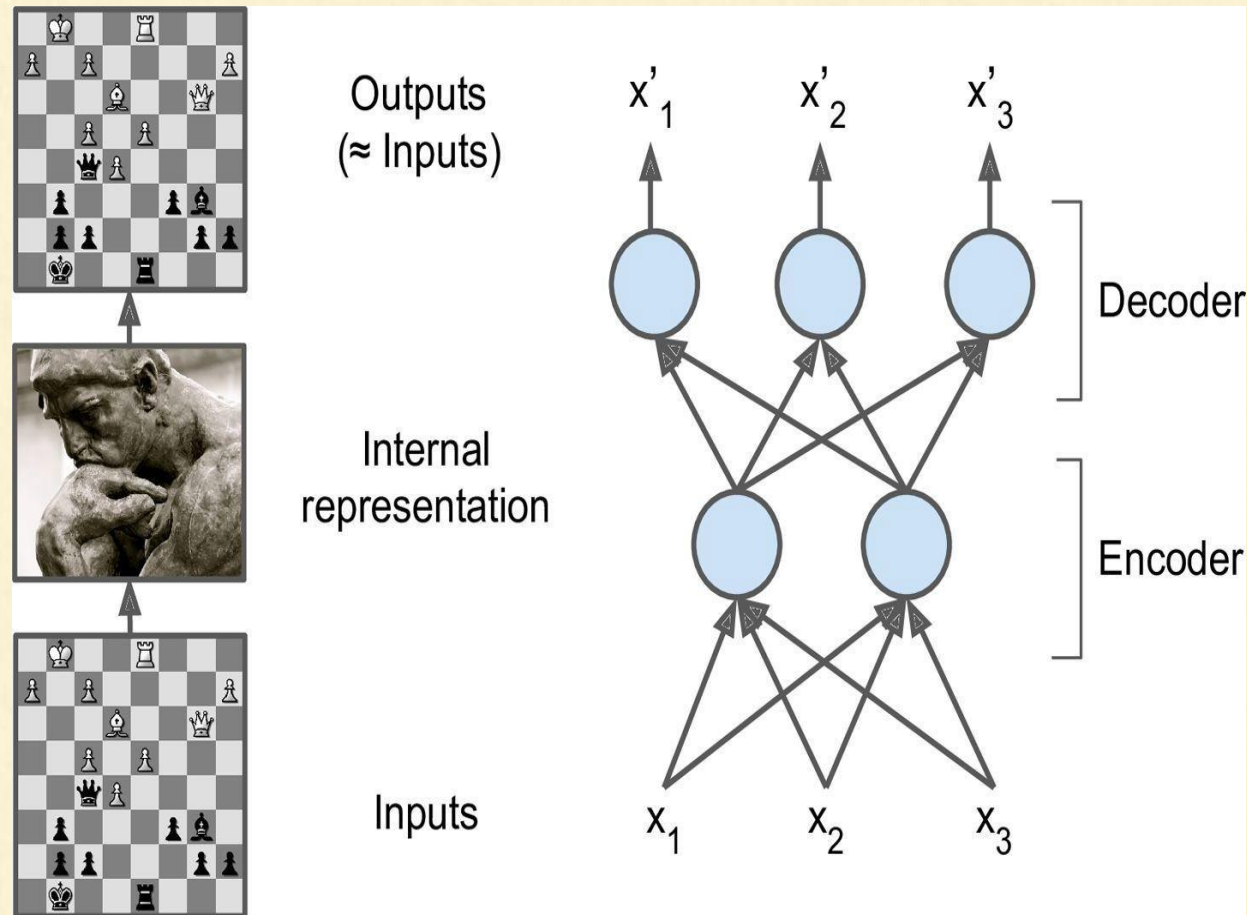
Composition of Autoencoder



Because the internal representation has a lower dimensionality than the input data, **it is 2D instead of 3D**, the autoencoder is said to be **undercomplete**

Efficient Data Representations

Composition of Autoencoder



- An **undercomplete** autoencoder cannot trivially copy its inputs to the **codings**, yet it must find a way to output a copy of its inputs
- It is forced to learn the most important features in the input data and drop the unimportant ones

Autoencoders for dimensionality reduction

PCA with an Undercomplete Linear Autoencoder

If the autoencoder uses **only linear activations** and the cost function is the **Mean Squared Error (MSE)**, then it can be shown that it ends up performing **Principal Component Analysis**

Now we will build a simple linear autoencoder to perform PCA on a 3D dataset, projecting it to 2D

PCA with an Undercomplete Linear Autoencoder

```
>>> import tensorflow as tf
>>> from tensorflow.contrib.layers import fully_connected
>>> n_inputs = 3 # 3D inputs
>>> n_hidden = 2 # 2D codings
>>> n_outputs = n_inputs
>>> learning_rate = 0.01
>>> X = tf.placeholder(tf.float32, shape=[None, n_inputs])
>>> hidden = fully_connected(X, n_hidden, activation_fn=None)
>>> outputs = fully_connected(hidden, n_outputs, activation_fn=None)
>>> reconstruction_loss = tf.reduce_mean(tf.square(outputs - X)) # MSE
>>> optimizer = tf.train.AdamOptimizer(learning_rate)
>>> training_op = optimizer.minimize(reconstruction_loss)
>>> init = tf.global_variables_initializer()
```

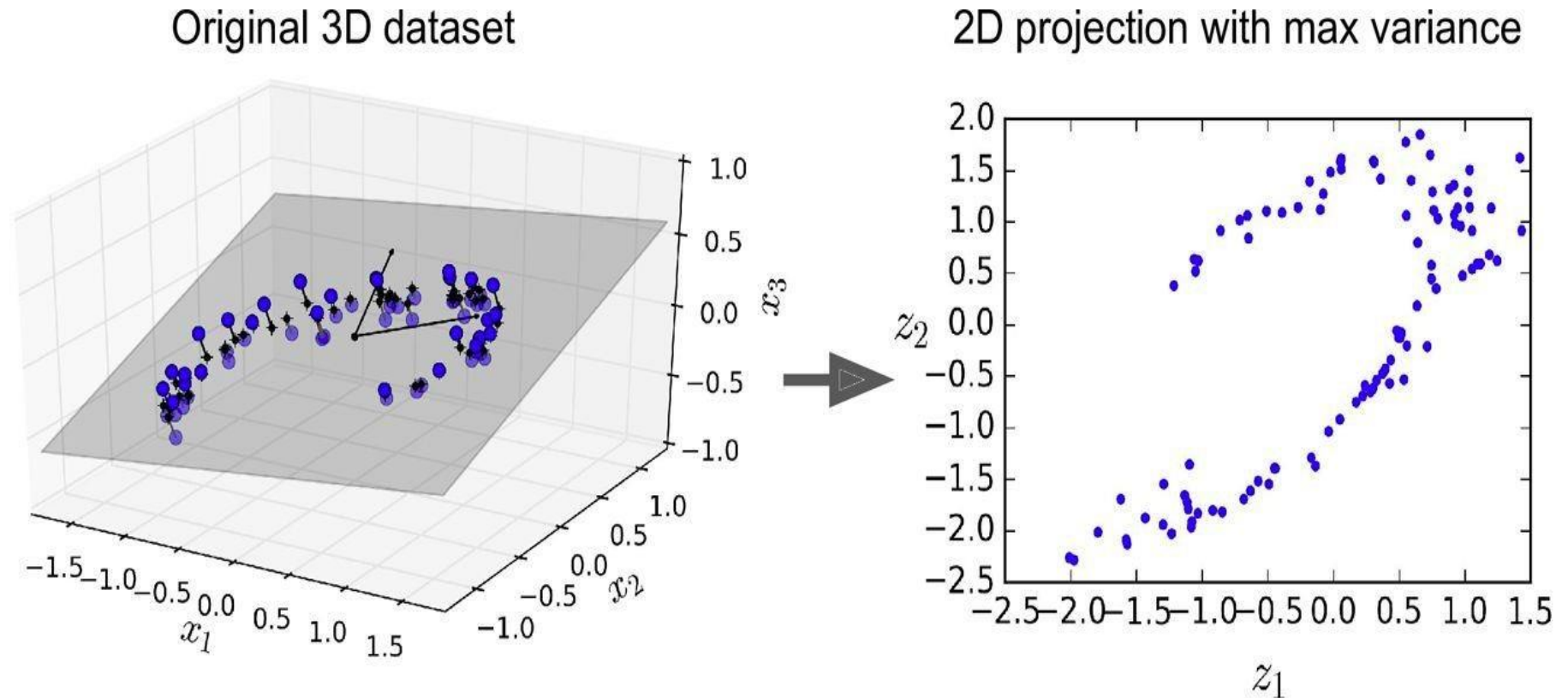
=

PCA with an Undercomplete Linear Autoencoder

The two things to note in the previous code are are:

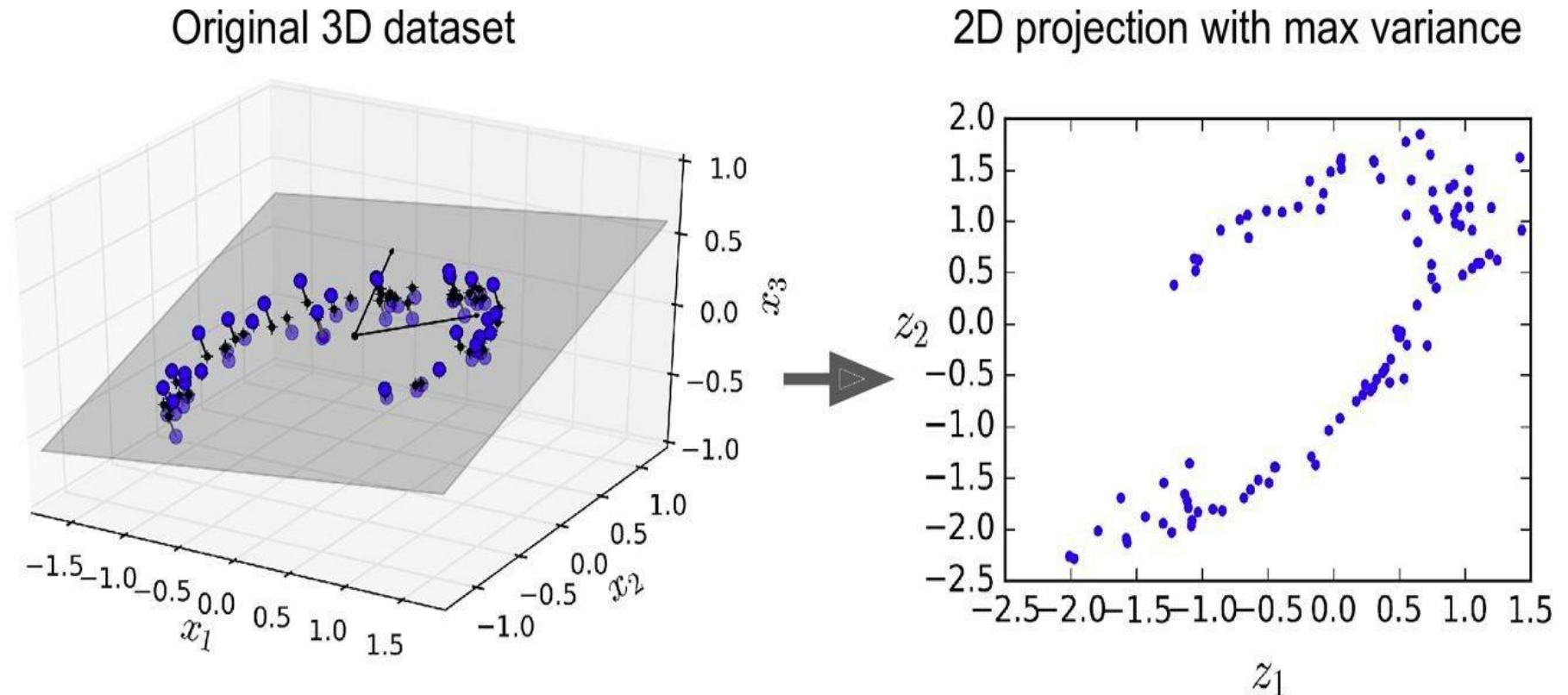
- The number of **outputs** is equal to the number of **inputs**
- To perform simple PCA, we set **activation_fn=None** i.e., all neurons are linear, and the cost function is the MSE.

PCA with an Undercomplete Linear Autoencoder



Above figure shows the original 3D dataset (at the left) and the output of the autoencoder's hidden layer (i.e., the coding layer, at the right)

PCA with an Undercomplete Linear Autoencoder



As you can see, the autoencoder found the best 2D plane to project the data onto, preserving as much variance in the data as it could just like PCA

Types of Autoencoders

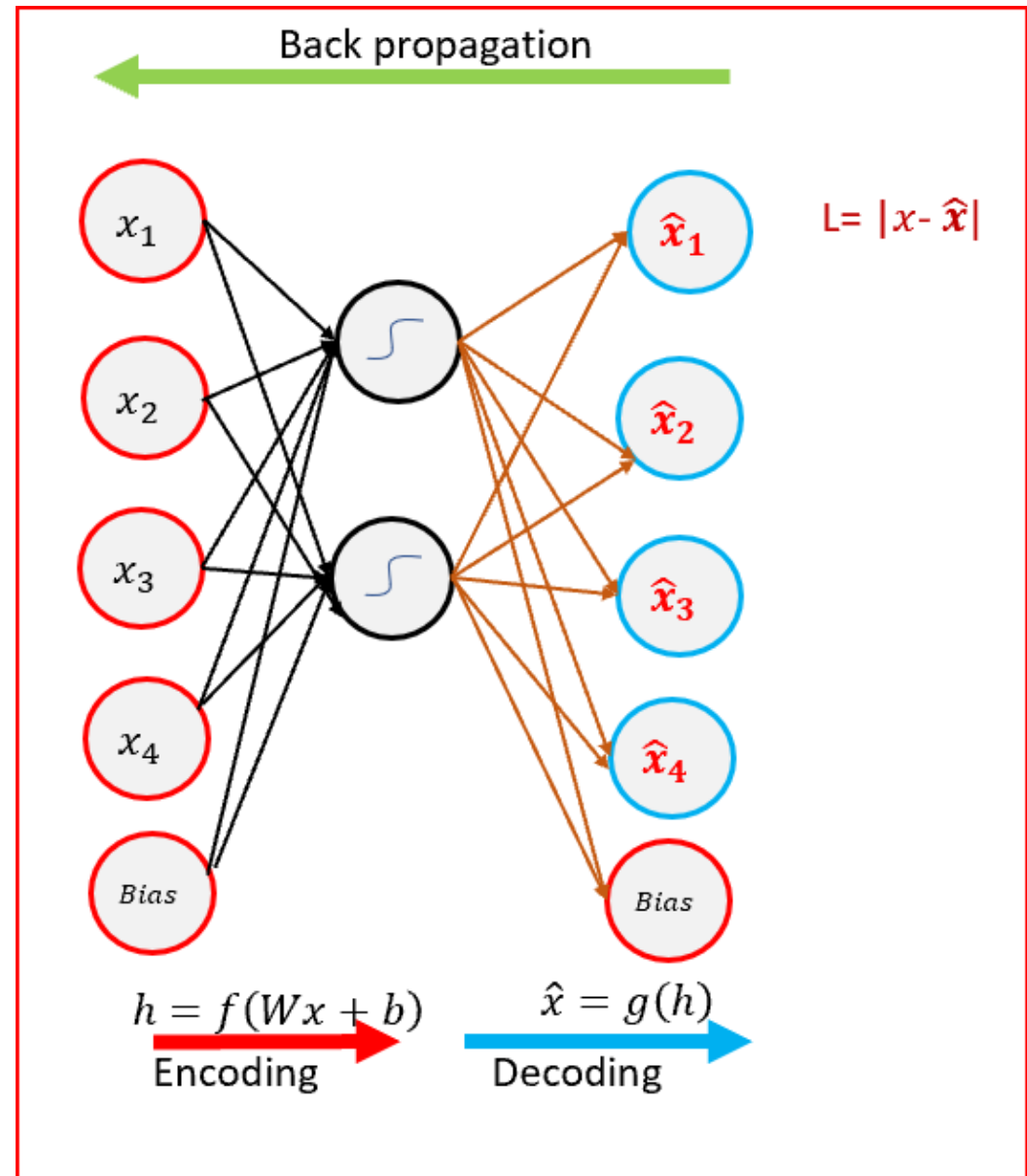
- Autoencoders encodes the input values x using a function f . Then decodes the encoded values $f(x)$ using a function g to create output values identical to the input values.
- Autoencoder objective is to minimize reconstruction error between the input and output. This helps autoencoders to learn important features present in the data. When a representation allows a good reconstruction of its input then it has retained much of the information present in the input.
- **What are different types of Autoencoders?**
 - Undercomplete Autoencoders
 - Multilayer autoencoder
 - Denoising Autoencoders (DAE)
 - Sparse Autoencoders

Undercomplete Autoencoders

- Undercomplete autoencoders have a smaller dimension for hidden layer compared to the input layer. This helps to obtain important features from the data.
- Objective is to minimize the loss function by penalizing the $g(f(x))$ for being different from the input x .

$$L = |x - \hat{x}|$$
$$L = |x - g(f(x))|$$

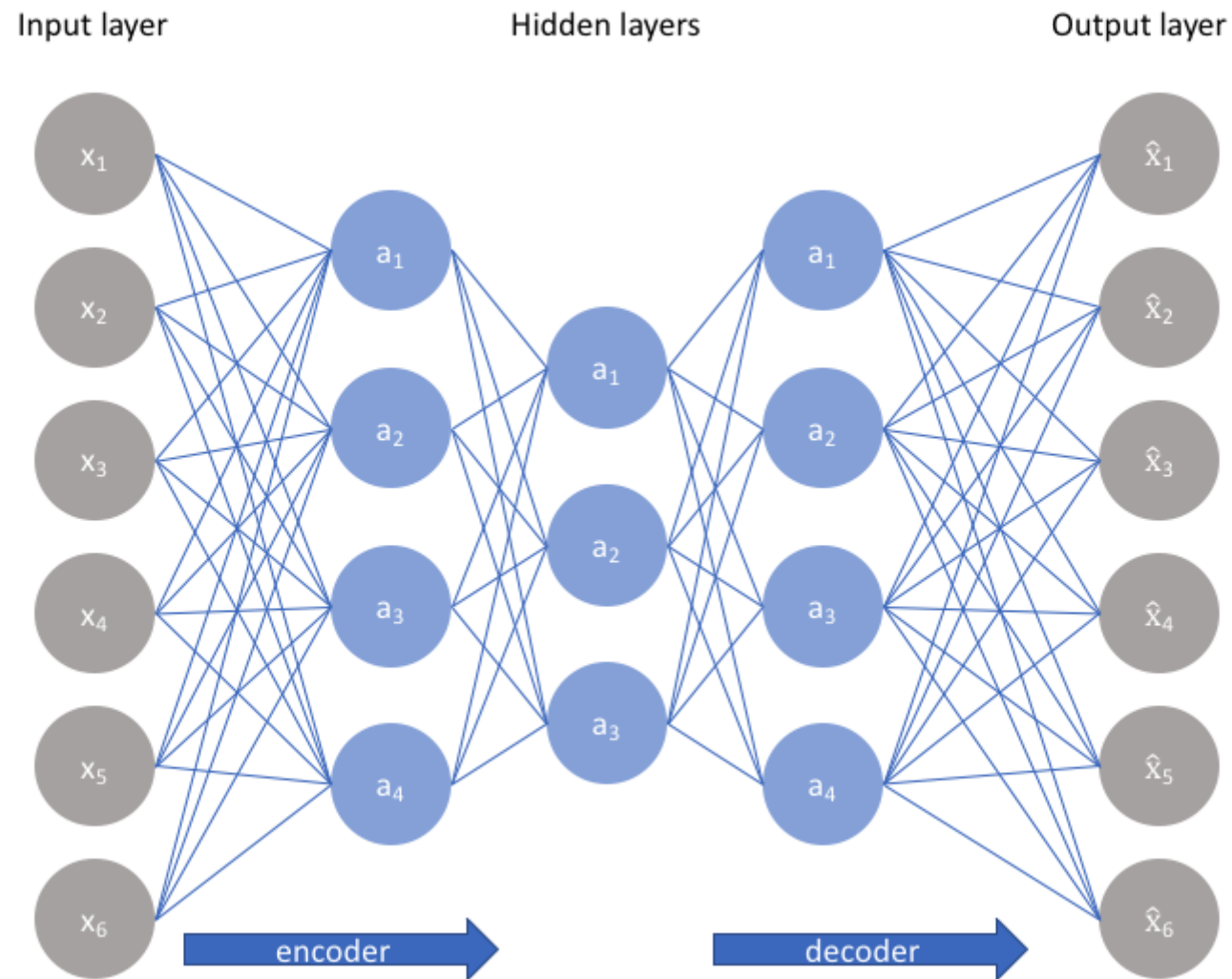
- When decoder is linear and we use a mean squared error loss function then undercomplete autoencoder generates a reduced feature space similar to [PCA](#).
- We get a powerful nonlinear generalization of PCA when encoder function f and decoder function g are non linear.



Undercomplete Autoencoders

```
1  input_size = 784
2  hidden_size = 64
3  output_size = 784
4
5  x = Input(shape=(input_size,))
6
7  # Encoder
8  h = Dense(hidden_size, activation='relu')(x)
9
10 # Decoder
11 r = Dense(output_size, activation='sigmoid')(h)
12
13 autoencoder = Model(input=x, output=r)
14 autoencoder.compile(optimizer='adam', loss='mse')
```

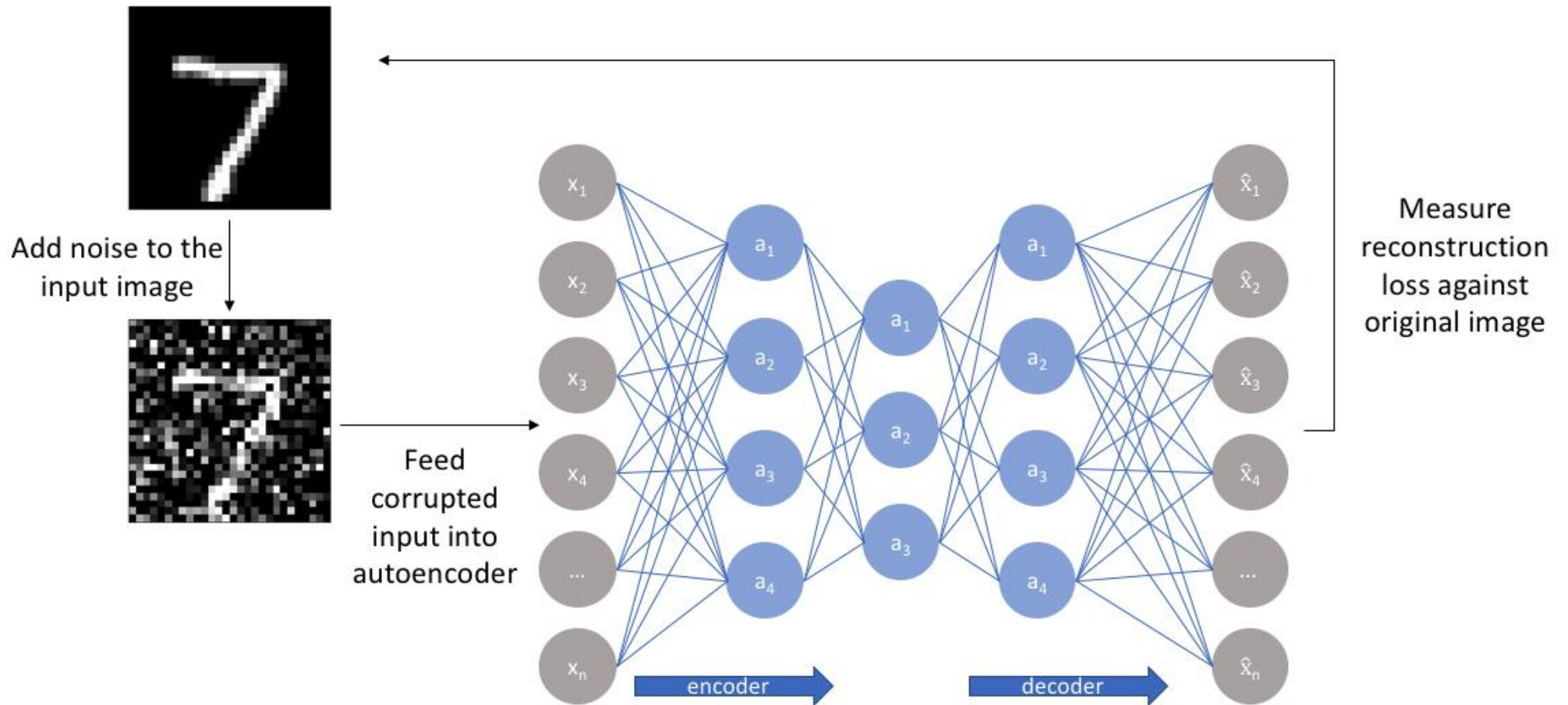
Multilayer Autoencoders



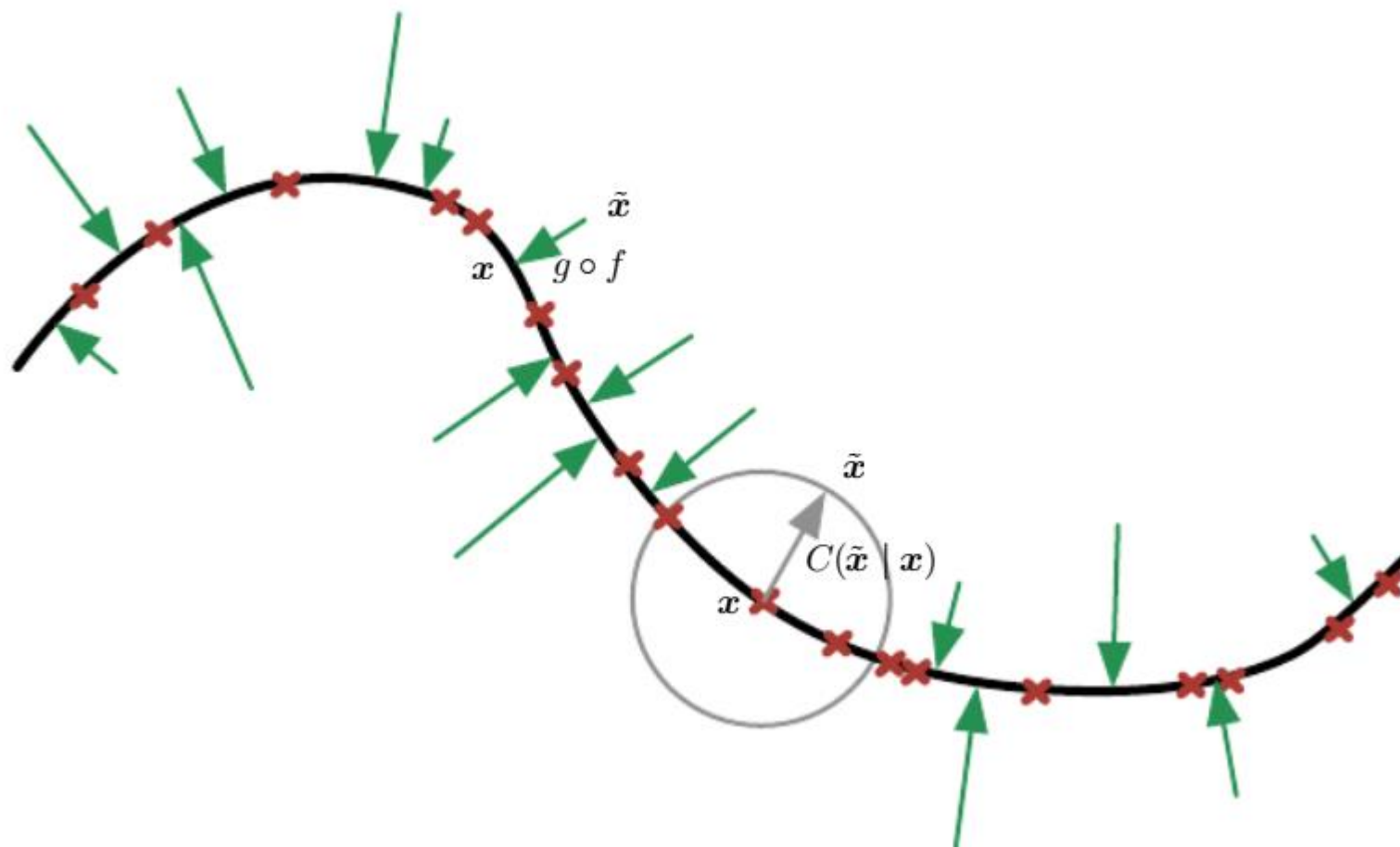
Multilayer Autoencoders

```
1  input_size = 784
2  hidden_size = 128
3  code_size = 64
4
5  x = Input(shape=(input_size,))
6
7  # Encoder
8  hidden_1 = Dense(hidden_size, activation='relu')(x)
9  h = Dense(code_size, activation='relu')(hidden_1)
10
11 # Decoder
12 hidden_2 = Dense(hidden_size, activation='relu')(h)
13 r = Dense(input_size, activation='sigmoid')(hidden_2)
14
15 autoencoder = Model(input=x, output=r)
16 autoencoder.compile(optimizer='adam', loss='mse')
```

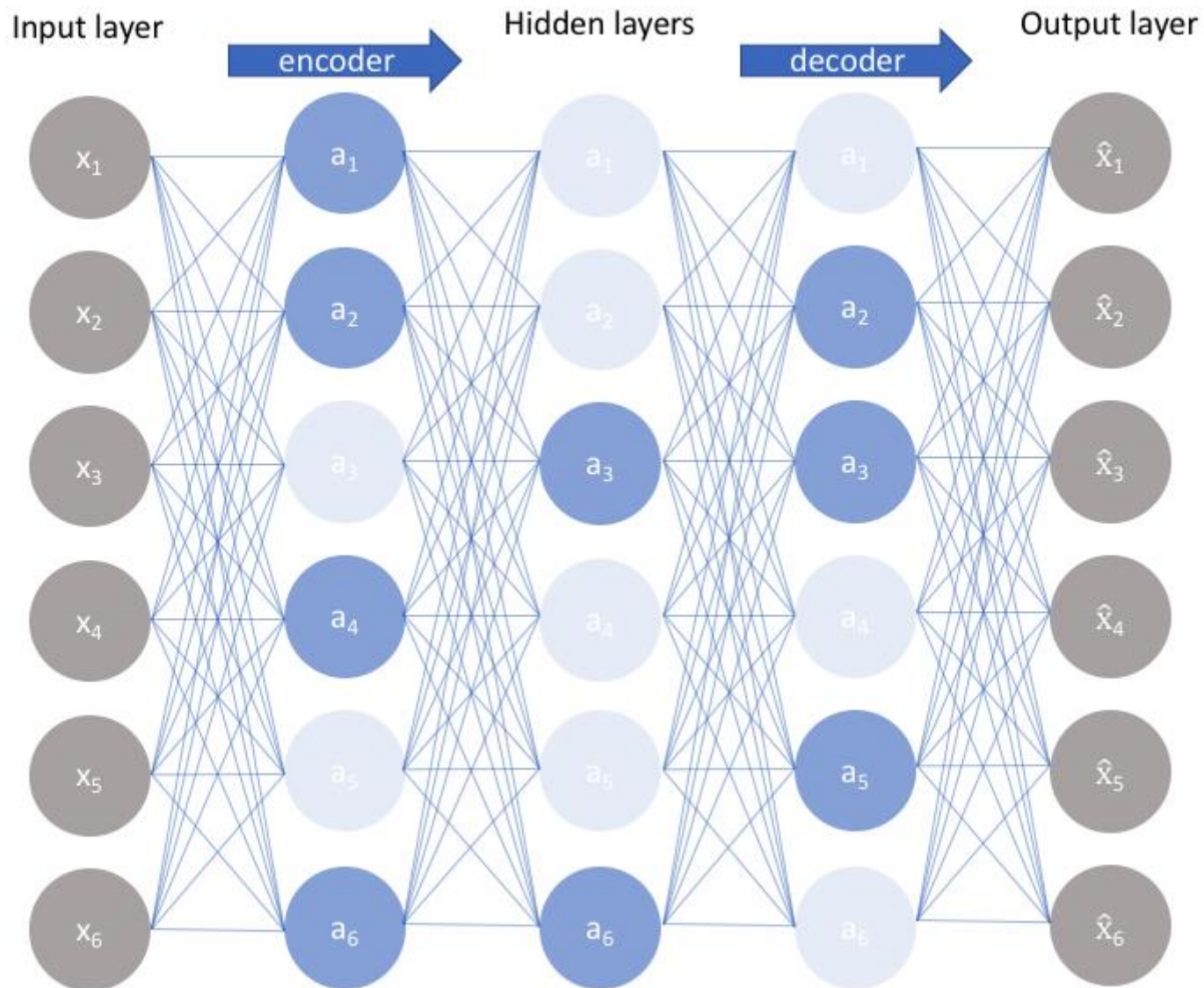
Denoising Autoencoders



Denoising Autoencoders



Sparse Autoencoders



Generative Adversarial Network

Probabilistic Generative Models



Density Estimation

- $Pr(\text{observation}) = Pr(\text{synthetic obs.})$

Probabilistic Generative Models



Density Estimation

- $Pr(\text{observation}) = Pr(\text{synthetic obs.})$

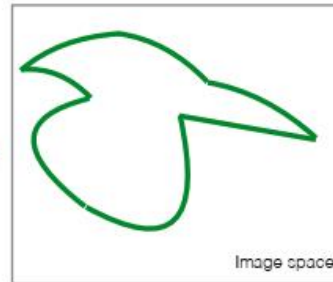
Motivation

Probability distributions:

Samples from the “real data distribution”

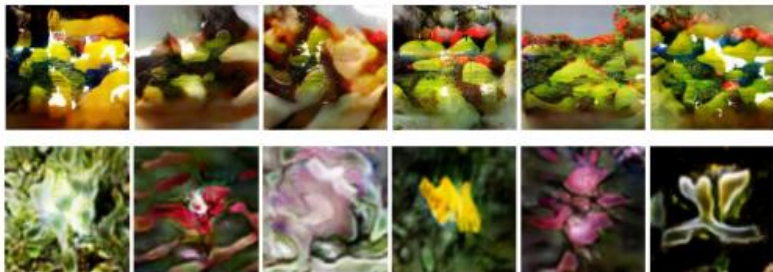


“real data distribution”

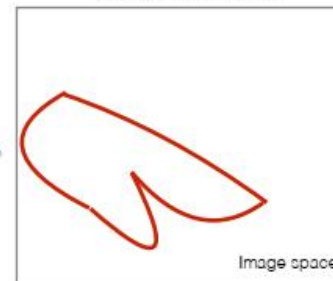


Goal

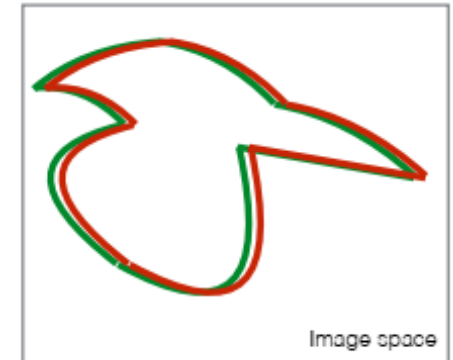
Samples from the “generated distribution”



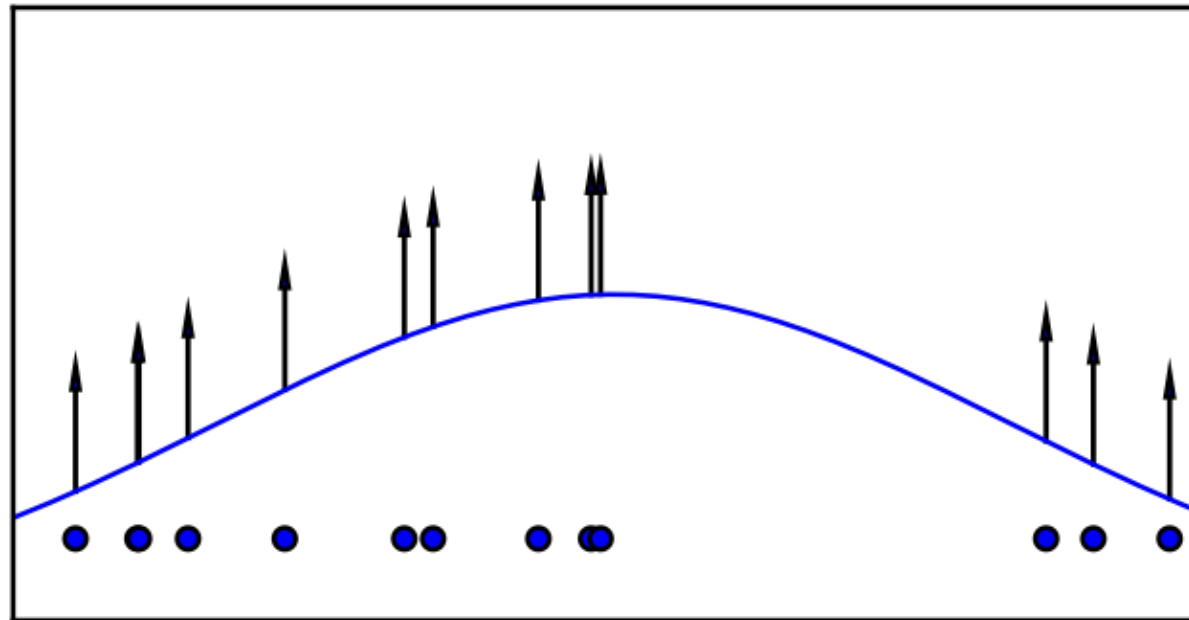
“generated distribution”



Matching distributions

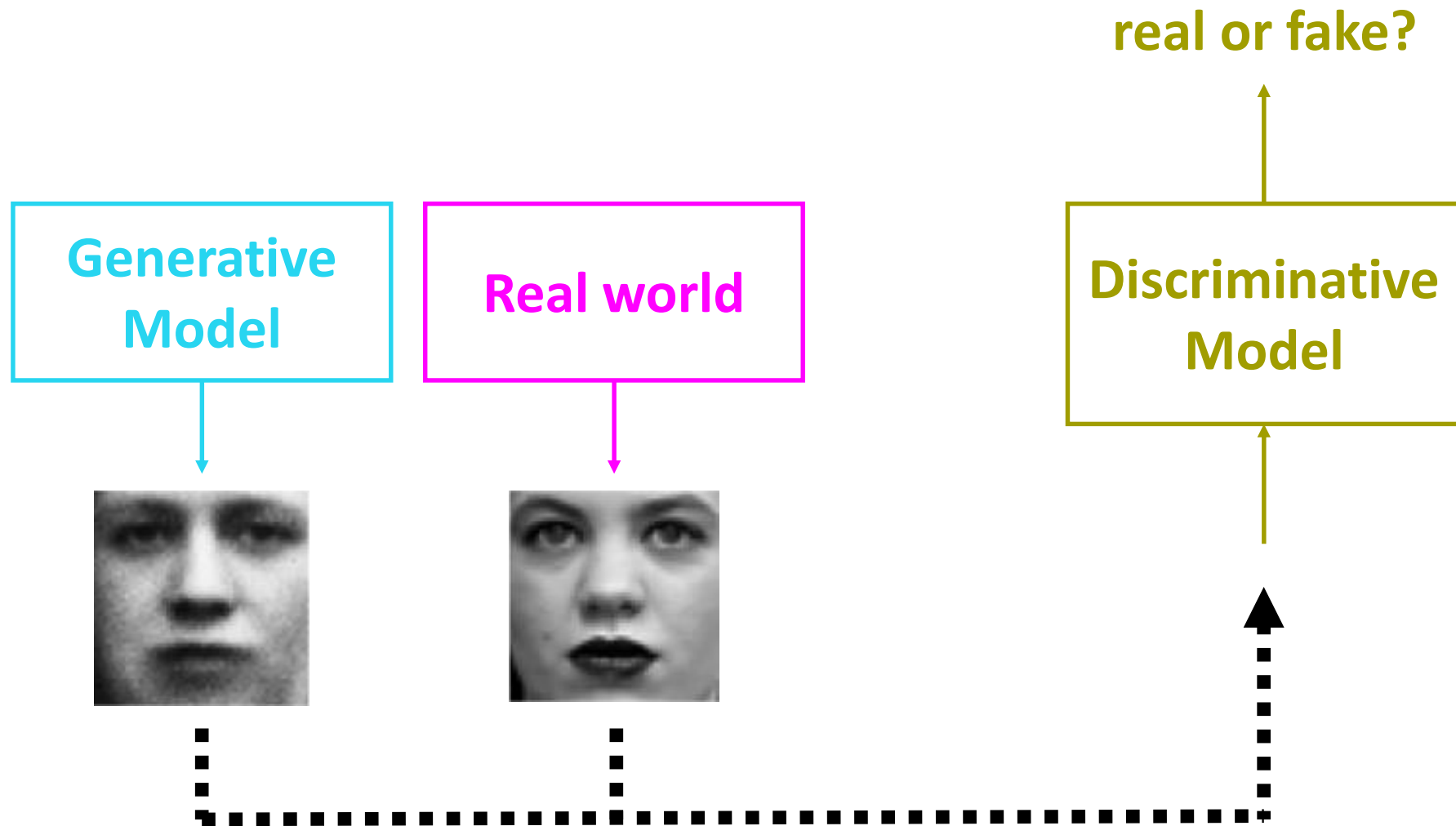


Maximum Likelihood Estimation



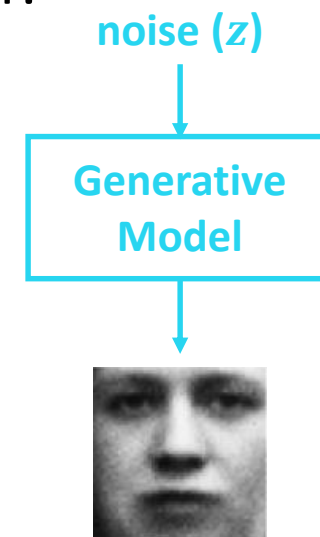
$$\theta^* = \arg \max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \log p_{\text{model}}(\mathbf{x} \mid \theta)$$

Adversarial Networks

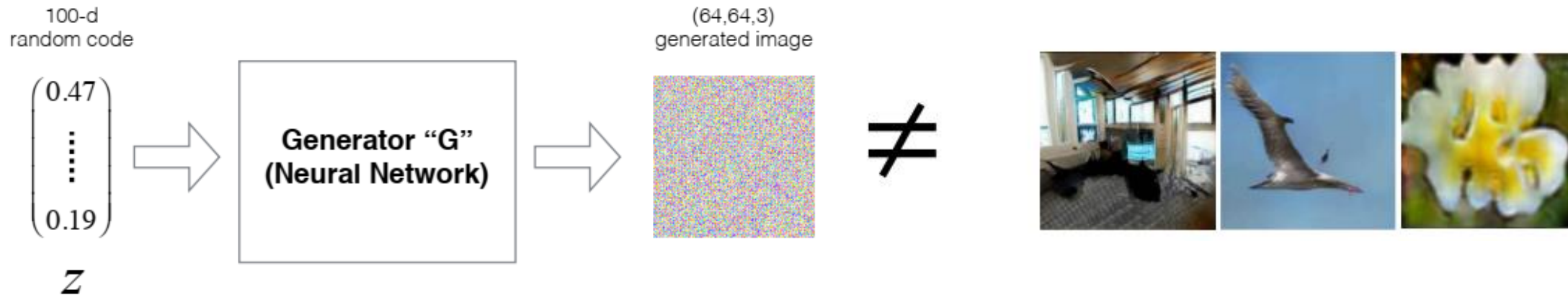


Generative Model

- How to make it generate different samples each time it is run?
 - input to model is noise
- Generative model as a neural network
 - computes $x = G(z|\theta)$
 - differentiable
 - does not have to be invertible
 - z typically has very high dimensionality (higher than x)



Generator/Discriminator Game



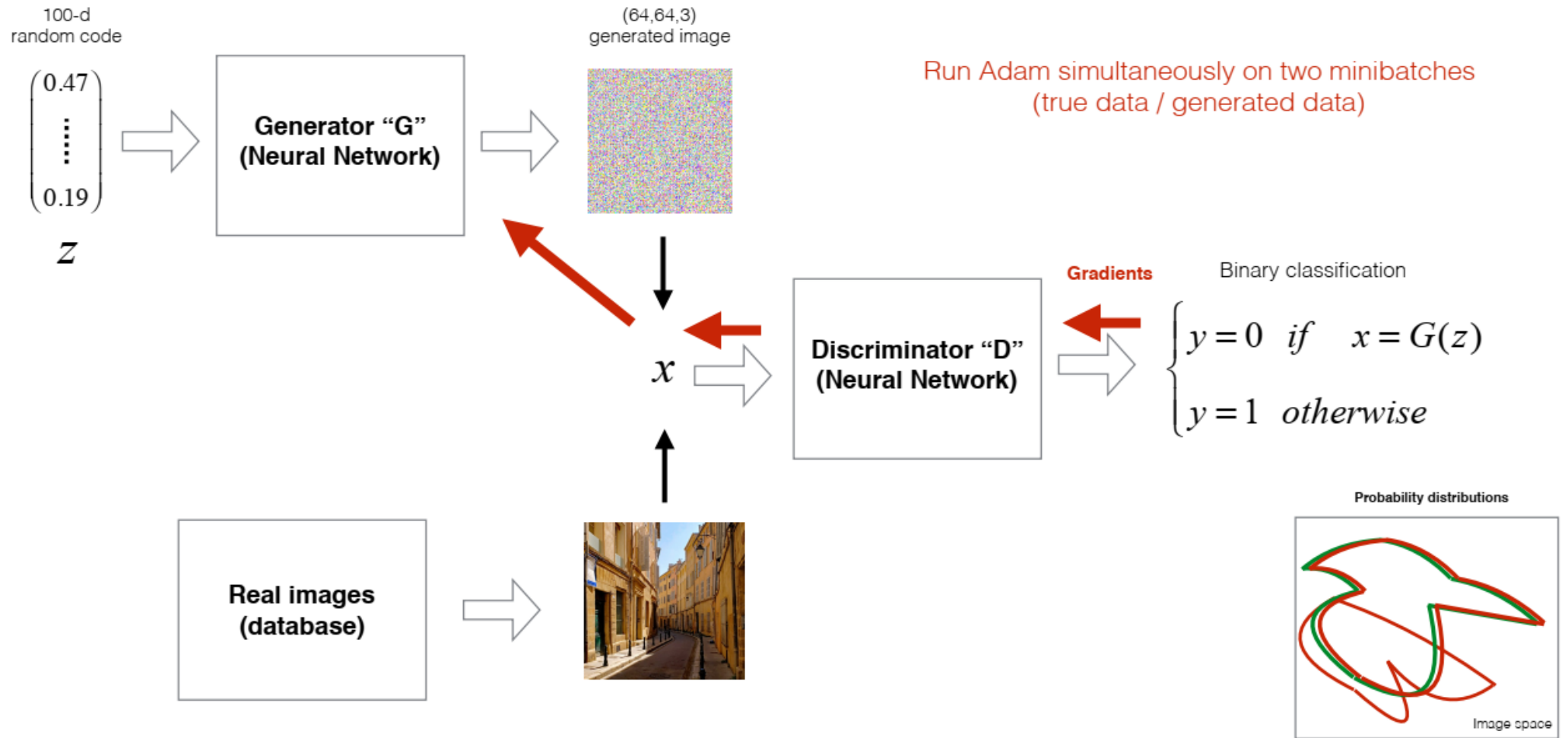
How can we train G to generate images from the true data distributions?

Discriminative Model

- Think of it as a critic
 - a good critic can tell real from fake
- Discriminative model as a neural net
 - differentiable
 - computes $\mathbf{D}(\mathbf{x})$, with value 1 if real, 0 if fake

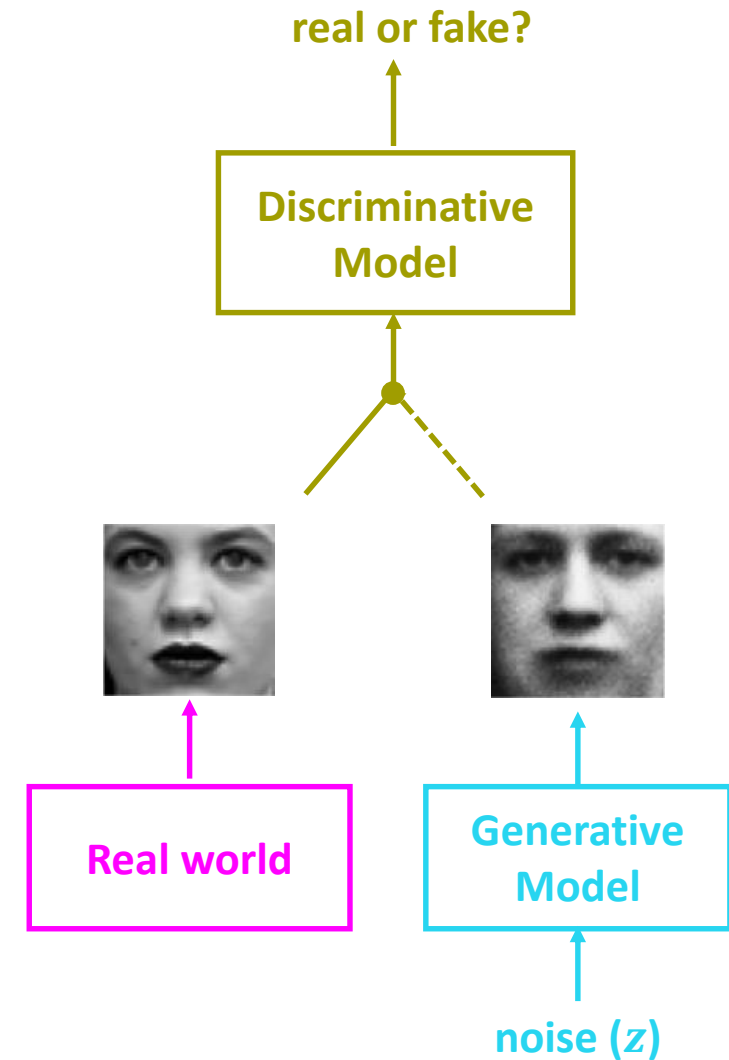


Generator/Discriminator Game



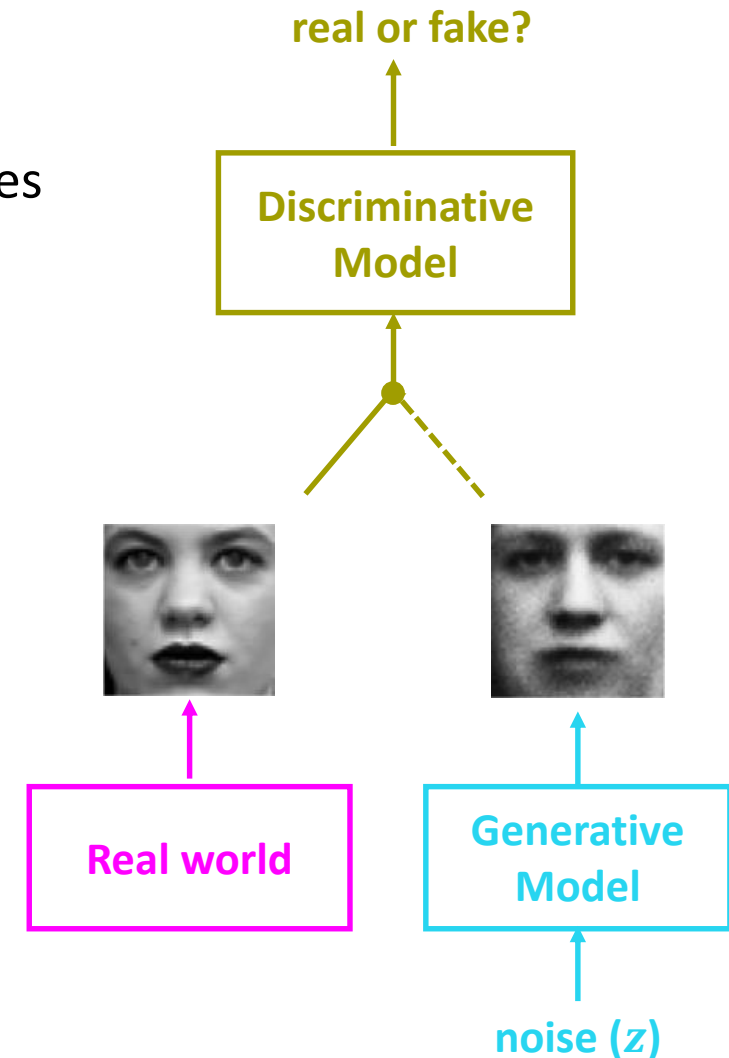
Training Procedure: Basic Idea

- G tries to fool D
- D tries not to be fooled
- Models are trained simultaneously
 - As G gets better, D has a more challenging task
 - As D gets better, G has a more challenging task
- Ultimately, we don't care about the D
 - Its role is to force G to work harder



Loss Functions

- Loss function for D
 - maximize the likelihood that model says 'real' to samples from the world and 'fake' to generated samples
 - $\mathcal{L}_D = -\frac{1}{2} \mathbb{E}_{x \sim \text{world}} \ln D(x) - \frac{1}{2} \mathbb{E}_z \ln (1 - D(G(z)))$
- What should the loss function be for G?
 - $\mathcal{L}_G = -\mathcal{L}_D$
- But because first term doesn't matter for G
 - $\mathcal{L}_D = \frac{1}{2} \mathbb{E}_z \ln (1 - D(G(z)))$
- Known as a minimax procedure



Generator/Discriminator Game

Training procedure, we want to minimize:

Labels: $\begin{cases} y_{real} & \text{is always 1} \\ y_{gen} & \text{is always 0} \end{cases}$

- The cost of the discriminator

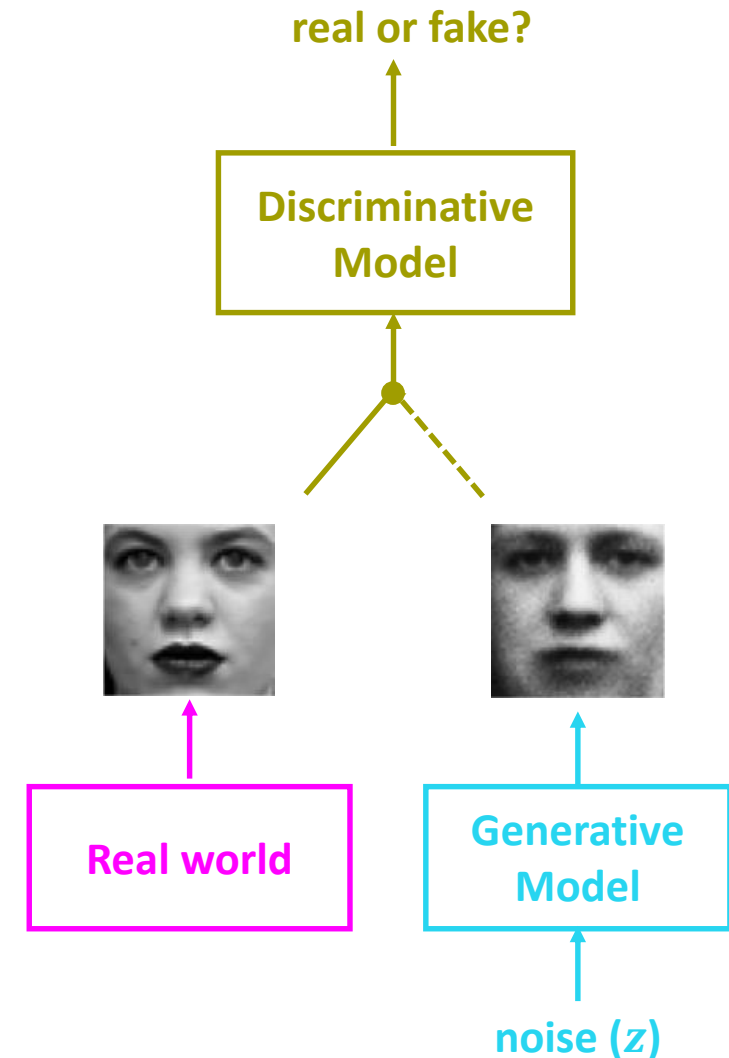
$$J^{(D)} = \underbrace{-\frac{1}{m_{real}} \sum_{i=1}^{m_{real}} y_{real}^{(i)} \cdot \log(D(x^{(i)}))}_{\text{cross-entropy 1: "D should correctly label real data as 1"}} - \underbrace{\frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} (1 - y_{gen}^{(i)}) \cdot \log(1 - D(G(z^{(i)})))}_{\text{cross-entropy 2: "D should correctly label generated data as 0"}}$$

- The cost of the generator

$$J^{(G)} = -J^{(D)} = \frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} \log(1 - D(G(z^{(i)}))) \quad \text{"G should try to fool D: by minimizing the opposite of what D is trying to minimize"}$$

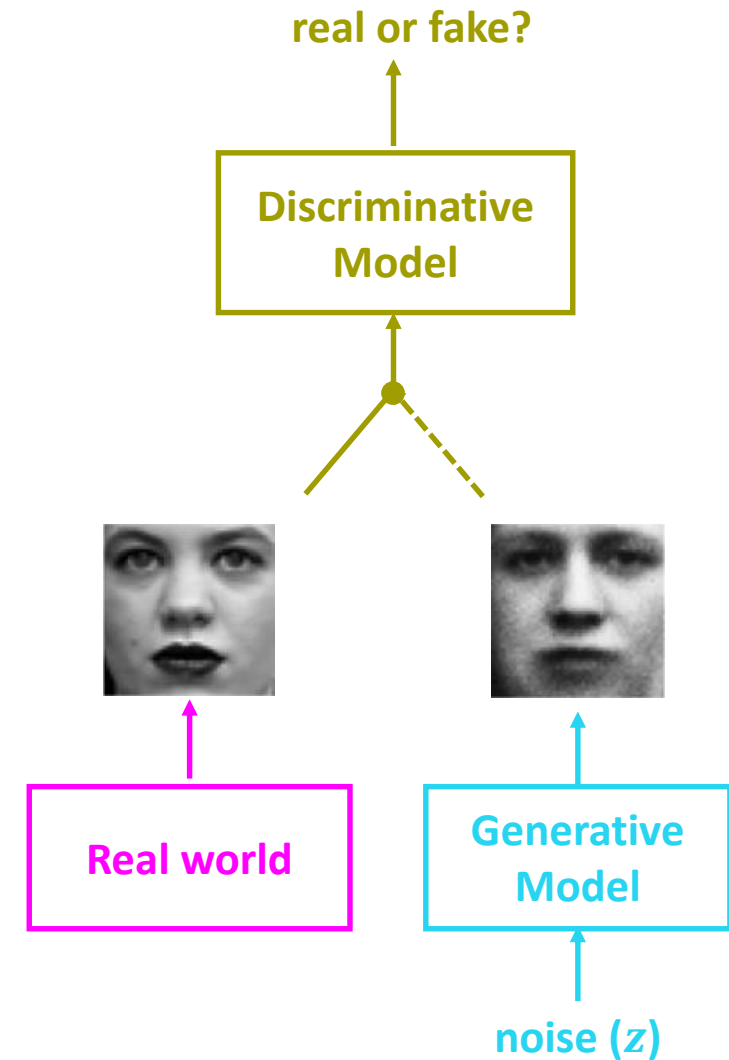
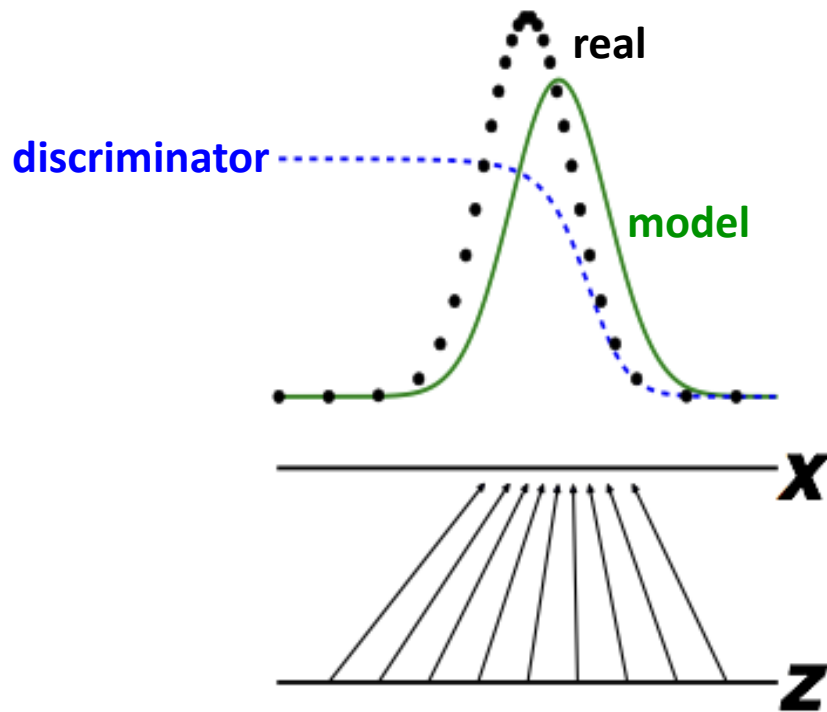
Training Procedure

- Train both models simultaneously via stochastic gradient descent using minibatches consisting of
 - some generated samples
 - some real-world samples
- Training of D is straightforward
- Error for G comes via back propagation through D
 - Two ways to think about training
 - (1) freeze D weights and propagate $\mathcal{L}_G$ through D to determine $\partial \mathcal{L}_G / \partial x$
 - (2) Compute $\partial \mathcal{L}_D / \partial x$ and then $\partial \mathcal{L}_G / \partial x = -\partial \mathcal{L}_D / \partial x$
- D can be trained without altering G, and vice versa
 - May want multiple training epochs of just D so it can stay ahead
 - May want multiple training epochs of just G because it has a harder task



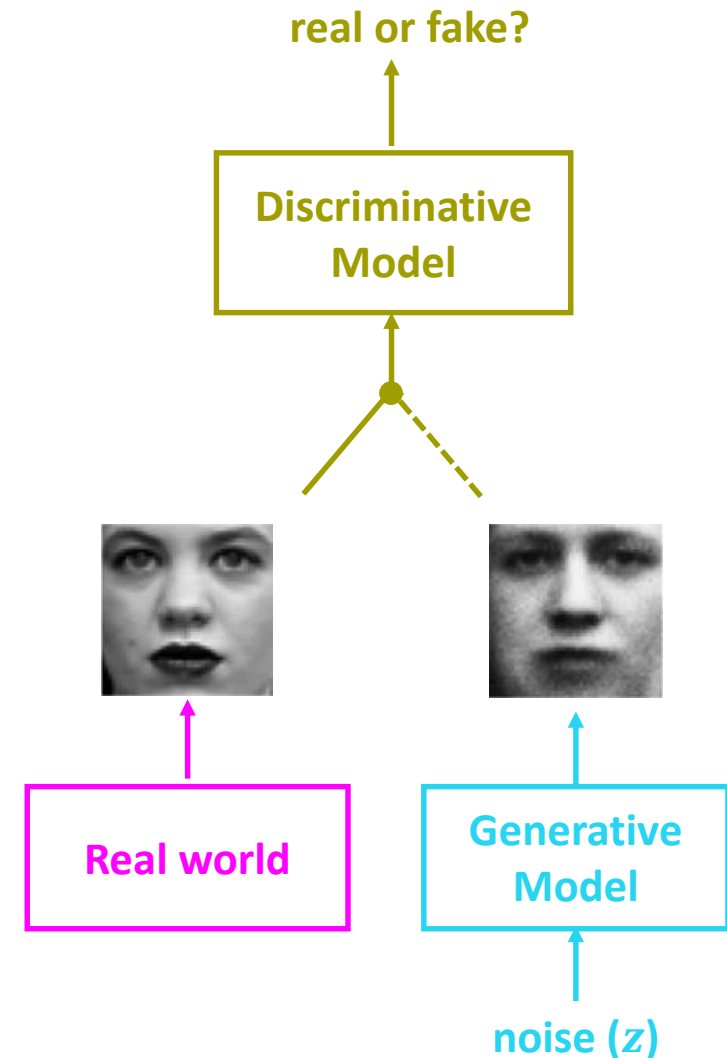
The Discriminator Has a Straightforward Task

- D has learned when
 - $D(x) = \frac{Pr(\text{real}|x)}{Pr(x|\text{real}) + Pr(x|\text{synthesized})}$



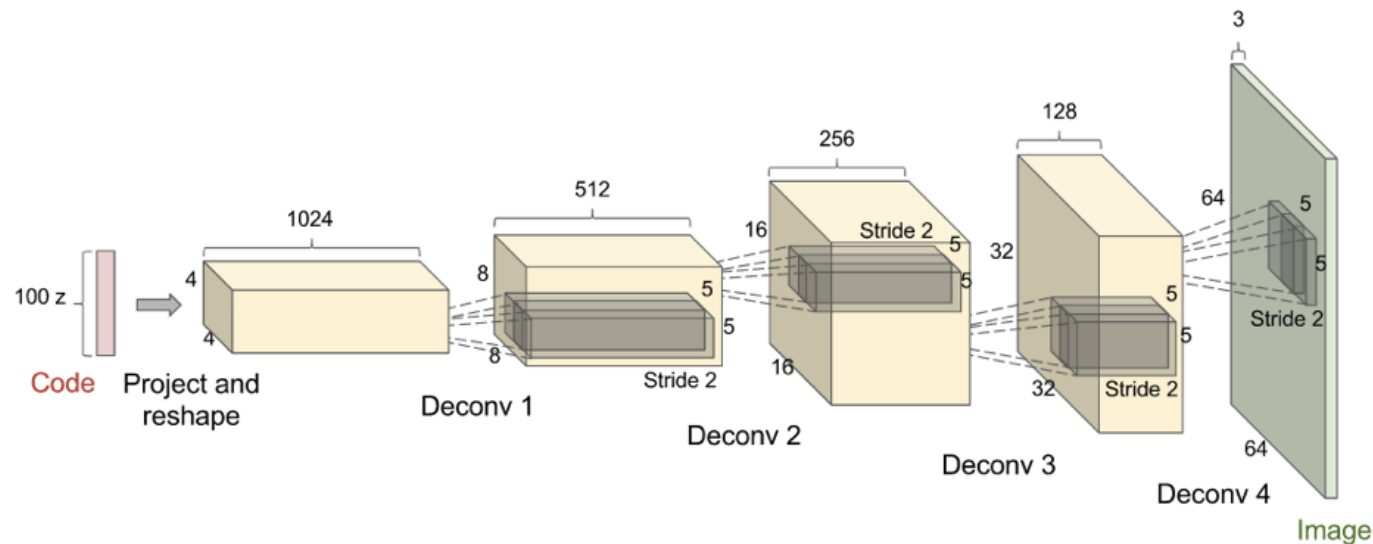
Three Reasons That It's a Miracle GANs Work

- G has a reinforcement learning task
 - it knows when it does good (i.e., fools D) but it is not given a supervised signal
 - reinforcement learning is hard
 - back prop through D provides G with a supervised signal; the better D is, the better this signal will be
- Can't describe optimum via a single loss
 - Will there be an equilibrium?
- D is seldom fooled
 - but G still learns because it gets a gradient telling it how to change in order to do better the next round.



Deconvolutional GANs (DCGAN)

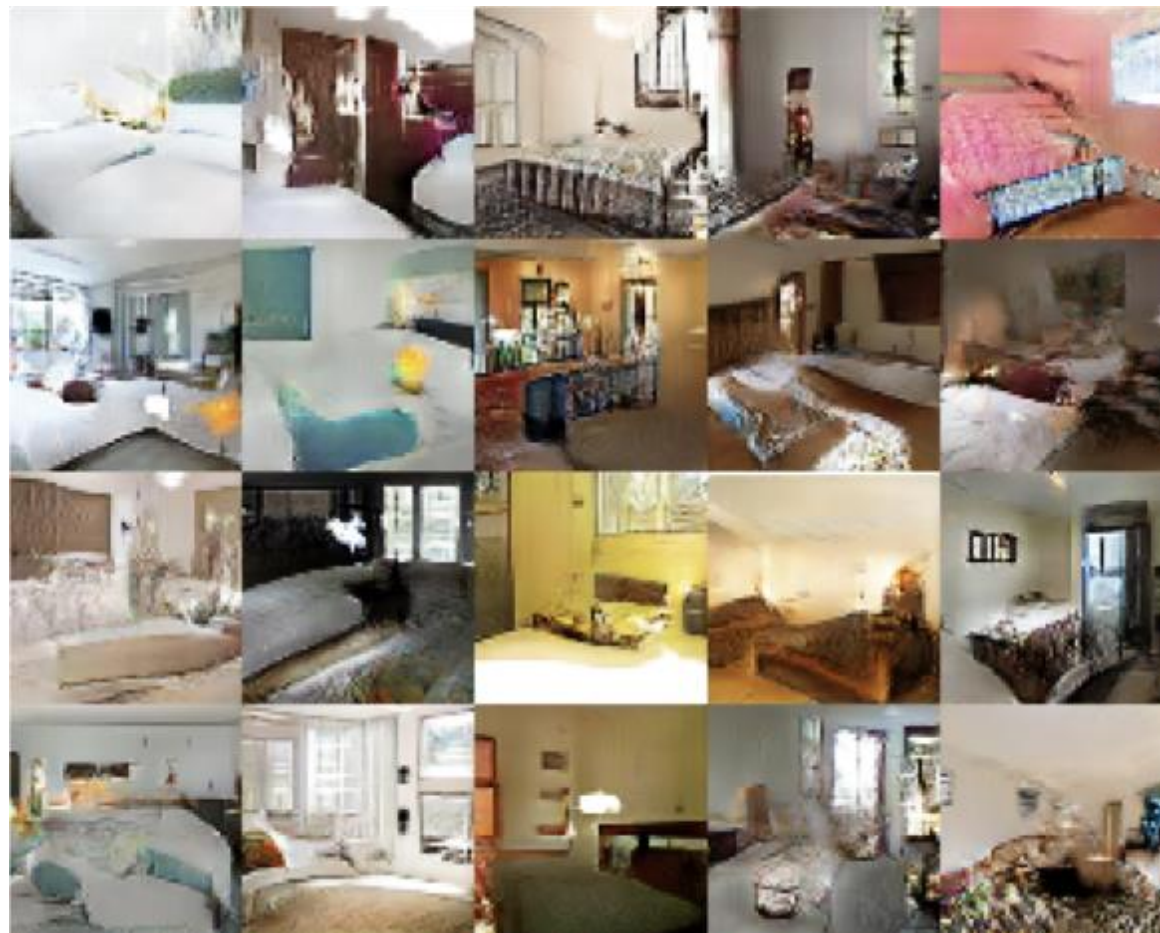
(Radford et al., 2015)



Batch normalization important here, apparently

Deconvolutional GANs (DCGAN)

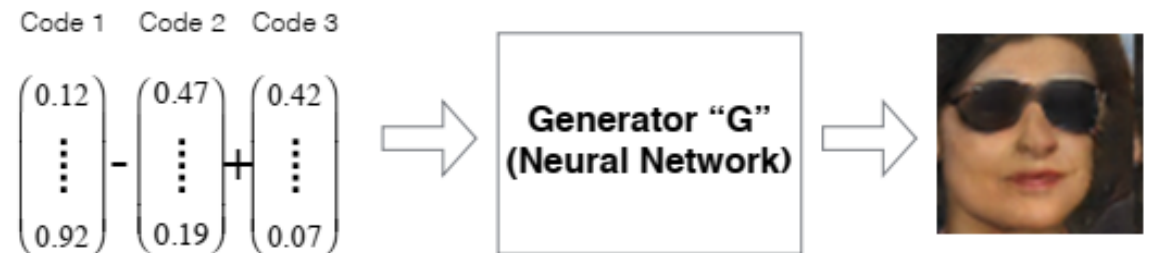
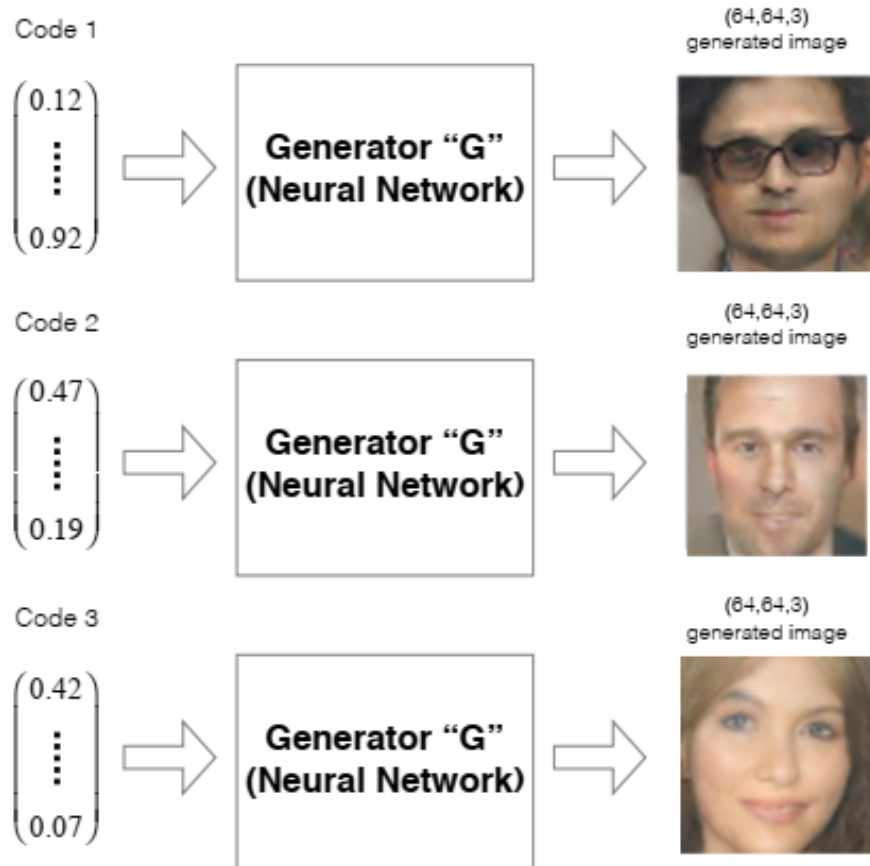
(Radford et al., 2015)



- Based on LSUN data set
 - 10 scene categories
 - 20 object categories
- [ArXiv 1506.03365](https://arxiv.org/abs/1506.03365)

Nice Results

Operation on codes



Man with glasses - man + woman = woman with glasses

Nice Results

Image Generation:

Samples from the “generated distribution”

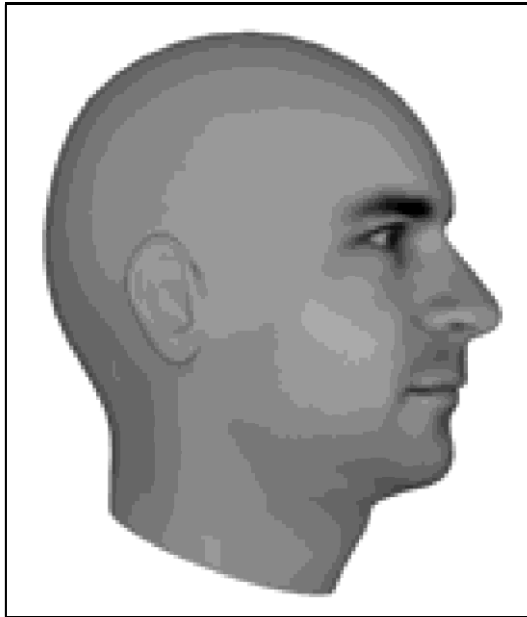


[Zhang et al. (2017): StackGAN++]

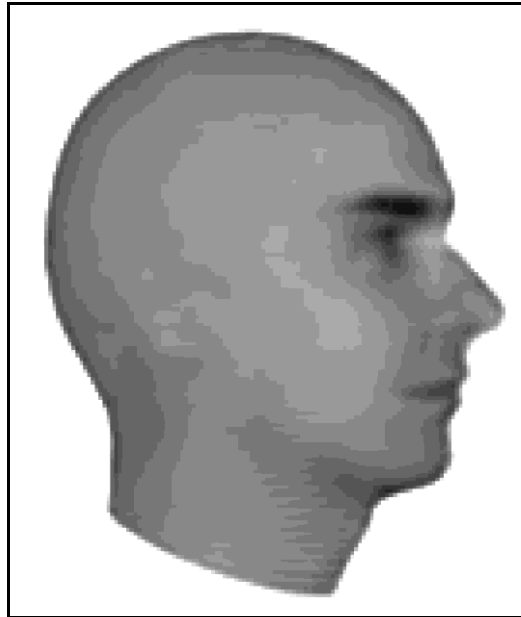
Beyond Labels: Providing Images as Input to Generator:

Next Video Frame Prediction (Lotter et al., 2016)

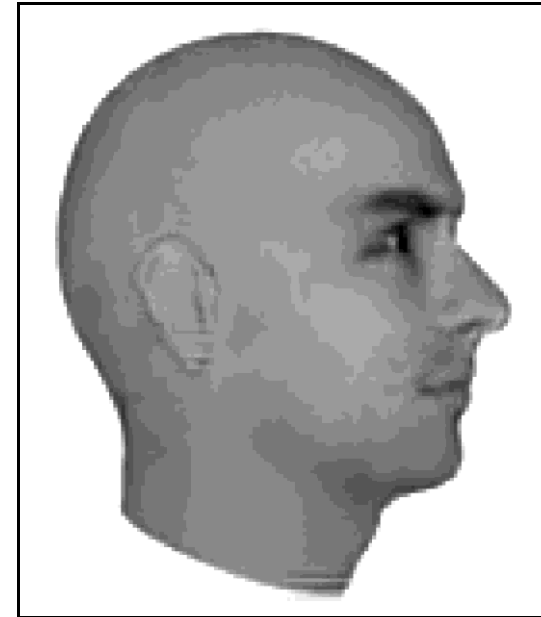
Ground Truth



MSE



Adversarial



- MSE tends to produce blurry images on any task
 - when you can't predict well, predict the expectation

Beyond Labels: Providing Images as Input to Generator:

Image Super-Resolution (Ledig et al., 2016)

